



Tiffin

The Complete Beginner-to-Interview Guide

Every language used in this app, taught from zero — then the entire codebase, explained line by line — then an interview-ready Q&A guide.

Prepared: September 2026

Audience: Complete beginners with zero programming background

How to use this document: Read Part 1 top to bottom even if some of it feels slow — it builds the vocabulary Part 2 assumes you already have. Part 2 quotes real code from this project and explains every line. Part 4 is for interview prep once you understand the rest.

Table of Contents

PART 1 — LEARN THE LANGUAGES (FROM ZERO)

1. How the Web Works
2. HTML — The Skeleton
3. CSS — The Styling
4. JavaScript Part A — Values, Variables, Operators
5. JavaScript Part B — Control Flow (if/else, loops)
6. JavaScript Part C — Functions
7. JavaScript Part D — Arrays & Objects
8. JavaScript Part E — Modern Syntax (arrow fns, destructuring, spread)
9. JavaScript Part F — The DOM & Events
10. JavaScript Part G — Event Delegation, Closures, IIFEs
11. JavaScript Part H — Async Code & Promises
12. JSON — The Data Format
13. Browser Storage — localStorage
14. Browser APIs — Camera, Files, Clipboard
15. Python — Just Enough to Read build.py
16. Git & GitHub Actions — Version Control and CI/CD
17. Capacitor — Turning a Website into a Phone App

PART 2 — THE APP, EXPLAINED END TO END

18. Project Tour — Every File and What It's For
19. The Build Pipeline — build.py Deep Dive
20. shell.html Deep Dive
21. app.css Deep Dive
22. food_db.json Deep Dive
23. app.js Part 1 — Setup & The Shape of the Data
24. app.js Part 2 — Storage & Security
25. app.js Part 3 — Utility & Math Functions
26. app.js Part 4 — The Diary & Meal System
27. app.js Part 5 — Energy Calculations (BMR/TDEE)
28. app.js Part 6 — The Rendering System
29. app.js Part 7 — The Today (Home) View
30. app.js Part 8 — The Food Picker Flow
31. app.js Part 9 — Custom Foods & Custom Meals
32. app.js Part 10 — Progress View & Charts
33. app.js Part 11 — Profile View

- 34. app.js Part 12 — Barcode Scanning
- 35. app.js Part 13 — Export / Import / Theme
- 36. app.js Part 14 — THE EVENT SYSTEM (the app's heart)
- 37. app.js Part 15 — Startup

PART 3 — ARCHITECTURE & DESIGN DECISIONS

- 38. Why No Framework, No Backend?
- 39. Security: XSS, CSP, Prototype Pollution
- 40. The Full Data Flow, End to End

PART 4 — INTERVIEW GUIDE

- 41. The 30-Second Pitch
- 42. Core Technical Q&A;
- 43. Deep-Dive Design Questions
- 44. Glossary — Every Term Used in This Guide

PART 1

Learn the Languages, From Zero

Before we look at a single line of the Tiffin app, we need a shared vocabulary. This part explains every technology the app uses, assuming you've never programmed before. Don't skim it — Part 2 will constantly refer back to words defined here.

1. How the Web Works

A website is just **files** — text files, really — that a browser (Chrome, Safari, Firefox) downloads and turns into the page you see. There are three core file types, and each one does a different job:

- **HTML** files describe the *structure* — "there's a heading here, a button there, a list of items below it."
- **CSS** files describe the *appearance* — colors, spacing, fonts, layout.
- **JavaScript** files describe the *behavior* — what happens when you click a button, type in a box, or scroll.

Think of building a house: HTML is the frame and rooms, CSS is the paint and furniture arrangement, and JavaScript is the electricity and plumbing — the stuff that actually *does* things.

Normally, a website talks to a **server** — a remote computer — to fetch data (like your bank balance or a friend's photos). Tiffin is unusual: it has **no server at all**. Every file it needs is loaded once, and after that the app runs entirely inside your browser, saving your data on your own device. We'll see exactly how later — this is one of the most interesting design decisions in the whole project.

2. HTML — The Skeleton

HTML stands for **HyperText Markup Language**. It isn't a programming language — it has no logic, no math, no decisions. It's a *markup* language: you wrap text in **tags** to say what that text is.

2.1 Tags and elements

A tag looks like `<tagname>`. Most tags come in pairs: an opening tag and a closing tag (which has a forward slash). Everything between them is that element's content.

```
<p>This is a paragraph of text.</p>
<h1>This is a big heading</h1>
<button>Click me</button>
```

Some tags never wrap content and don't need a closing tag, like the line-break tag `<br>` or the meta-information tag `<meta>`.

2.2 Attributes

Tags can carry extra information called **attributes**, written inside the opening tag as `name="value"` pairs:

```
<button id="saveBtn" class="btn btn-primary" type="submit">Save</button>
```

- `id` — a unique name for one specific element, used to find it later.
- `class` — a label (that can be reused on many elements) mainly used by CSS to style groups of elements the same way.
- `type` — tag-specific meaning, e.g. a button's type or an input's kind (text, number, date...).

Tiffin's code uses one attribute constantly that isn't a standard HTML attribute: `data-action="..."`. Any attribute starting with `data-` is a "custom data attribute" — HTML explicitly allows you to invent your own, and JavaScript can read them. Tiffin uses these to label every clickable element with what it should *do* when clicked, e.g. `data-action="delete-entry"`. We'll see exactly how in Chapter 36.

2.3 The document skeleton

Every HTML page has the same basic shape:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Page title shown in the browser tab</title>
    <style>/* CSS can go here */</style>
  </head>
  <body>
    <!-- everything visible on the page goes here -->
    <script>/* JavaScript can go here */</script>
  </body>
</html>
```

- `<!doctype html>` tells the browser "this is a modern HTML document."
- `<head>` holds information *about* the page (title, styles, metadata) — nothing in here is directly visible.
- `<body>` holds everything the user actually sees.
- A `<script>` tag either contains JavaScript code directly, or points to a `.js` file via `src="..."`.

2.4 Common tags you'll see in this project

Tag	Meaning
<code><div></code>	A generic box/container with no special meaning — used purely for grouping and styling
<code></code>	Like <code><div></code> , but for inline (within a line of text) grouping
<code><button></code>	A clickable button
<code><input></code>	A field the user types into (or a checkbox, date picker, number field, etc.)
<code><select></code>	A dropdown menu
<code><form></code>	Groups related inputs together so they can be submitted as one unit
<code><svg></code>	Draws vector graphics (used here for all icons — no image files needed)
<code><canvas></code>	A blank drawing surface (used here by the charting library)
<code><video></code>	Plays a video stream (used here to show the live camera feed for barcode scanning)
<code><nav></code>	Marks a navigation menu
<code><label></code>	Text describing an input field

Note: Tiffin builds ALL of its HTML at runtime using JavaScript — there's barely any HTML written by hand. The one hand-written file, `shell.html`, is just an empty shell with a few placeholder containers (like `<div id="viewRoot"></div>`) that JavaScript fills in. This pattern is explained fully in Chapter 20.

3. CSS — The Styling

CSS stands for **Cascading Style Sheets**. It answers questions like: what color is this text? How much space is around this box? Should this be hidden on small screens?

3.1 The basic rule shape

```
selector {  
  property: value;  
  property: value;  
}  
  
/* Real example: */  
.btn-primary {  
  background: #FF6B4A;  
  color: white;  
  padding: 10px 18px;  
  border-radius: 999px;  
}
```

A **selector** picks which HTML elements the rule applies to. `.btn-primary` (dot prefix) selects every element with `class="btn-primary"`. `#saveBtn` (hash prefix) selects the one element with `id="saveBtn"`. A bare word like `button` selects every `<button>` tag on the page.

3.2 The box model

Every HTML element is treated as a rectangular box with four layers, from inside out: **content** (the text/image itself), **padding** (space inside the border), **border** (a line around the padding), and **margin** (space outside the border, separating it from other elements).

3.3 Flexbox and Grid — modern layout

Tiffin lays out almost everything with **Flexbox** (one-dimensional layout — a row or a column) and occasionally **Grid** (two-dimensional layout — rows AND columns at once).

```
.row {  
  display: flex;           /* children line up in a row */  
  align-items: center;     /* vertically centered */  
  justify-content: space-between; /* push children to opposite ends */  
  gap: 12px;              /* space between children */  
}  
  
.grid-2 {  
  display: grid;  
  grid-template-columns: 1fr 1fr; /* two equal-width columns */  
  gap: 14px;  
}
```

3.4 CSS variables (custom properties)

CSS lets you define your own named values, and reuse them everywhere, similar to a variable in programming. Tiffin defines its entire color palette this way at the top of `app.css`:

```
:root {
  --accent: #FF6B4A;
  --text: #241C12;
  --radius-m: 20px;
}

/* used elsewhere as: */
.btn-primary { background: var(--accent); }
```

The huge benefit: to support **dark mode**, Tiffin doesn't need to rewrite every single rule — it just redefines these same variable names inside a dark-mode block, and every rule that used `var(--accent)` automatically updates. We'll see this exact trick in Chapter 21.

3.5 Media queries — responsive design

A media query is a conditional rule that only applies when the browser window matches some condition, most commonly a maximum or minimum width — this is how a page adapts between desktop and mobile.

```
@media (max-width: 860px) {
  .sidenav { display: none; } /* hide the sidebar on small screens */
  .bottomnav { display: flex; } /* show a bottom tab bar instead */
}
```

3.6 Pseudo-classes

A pseudo-class targets an element in a particular *state*, written with a colon:

- `:hover` — while the mouse is over the element
- `:active` — while it's being clicked/pressed
- `:focus` — while a text input is selected/typing
- `:first-child` — if it's the first element inside its parent

4. JavaScript Part A — Values, Variables, Operators

JavaScript ("JS") is the one true **programming language** in this stack — it has logic, math, decisions, and memory. It's the only language capable of making the app actually *do* things rather than just look a certain way.

4.1 Variables — named storage

A variable is a labeled box that holds a value. JavaScript has three ways to create one:

```
const name = "Alice"; // cannot be reassigned later
let score = 100;      // CAN be reassigned later
score = 150;          // OK, because it was declared with let

var old = "avoid this"; // an older, quirky way – not used in this app
```

Tiffin uses `const` for almost everything, and `let` only for values that genuinely change over time (like which screen is currently showing). This is considered good modern style: it signals intent to future readers.

4.2 Data types

Type	Example	What it's for
Number	42, 3.14, -7	Any numeric value — JS has only one number type
String	'hello', "world"	Text, wrapped in quotes
Boolean	true / false	A yes/no, on/off value
Array	[1, 2, 3]	An ordered list of values
Object	{name: 'Alice'}	A collection of named values (key-value pairs)
null	null	"Intentionally nothing" — explicitly set to empty
undefined	undefined	"Nothing was ever set here" — JS's own default

Tiffin's food data is a perfect real example of an object nested inside an array of objects:

```
const food = {
  id: "rice-cooked-white",
  name: "Rice, cooked (Chawal, white)",
  kcal: 200,
  protein: 4.2,
  carbs: 44
};

const foodList = [food, anotherFood, yetAnotherFood]; // an array of objects
```

4.3 Operators

- **Arithmetic:** + - * / % (% is "remainder", e.g. 7 % 2 = 1)

- **Comparison:** === (equal), !== (not equal), > < >= <=
- **Logical:** && (AND — both sides must be true), || (OR — at least one side true), ! (NOT — flips true/false)
- **Assignment:** = (set), += (add and reassign), etc.

Note: Always use === instead of == in JavaScript. == silently converts types before comparing (so 0 == '0' is true, which causes bugs), while === requires an exact match. This codebase uses === throughout.

4.4 Template literals — building strings with data inside them

Backtick-quoted strings let you embed JavaScript expressions directly inside text using `${...}`. This is used *constantly* in Tiffin to build HTML:

```
const name = "chicken curry";
const kcal = 350;

// Old way:
const msg1 = "Added " + name + " (" + kcal + " kcal)";

// Template literal way (used everywhere in this app):
const msg2 = `Added ${name} (${kcal} kcal)`;

// Can even span multiple lines, which is how HTML gets built:
const html = `

<span>${name}</span>
  <span>${kcal} kcal</span>
</div>`;


```

5. JavaScript Part B — Control Flow

5.1 if / else — making decisions

```
if (age >= 18) {
  console.log("adult");
} else if (age >= 13) {
  console.log("teen");
} else {
  console.log("child");
}
```

5.2 The ternary operator — a compact if/else

Tiffin uses this shorthand everywhere inside template literals, because a full if/else statement can't be embedded inside a string expression:

```
const label = age >= 18 ? "adult" : "minor";

// Real example from the app (choosing an emoji fallback):
`Hi, ${name ? escapeHtml(name) + ' ■' : 'Welcome back ■'}`
```

5.3 Loops — repeating an action

```
// for loop: repeat a known number of times
for (let i = 0; i < 5; i++) {
  console.log(i); // prints 0,1,2,3,4
}

// while loop: repeat until a condition becomes false
let count = 0;
while (hasMoreItems) {
  count++;
}
```

In practice, this codebase almost never writes a raw for loop over arrays — it uses array methods like `.map()` and `.forEach()` instead (Chapter 7), which is the modern, preferred style.

5.4 Truthy and falsy values

In an if condition, JavaScript treats some non-boolean values as "falsy" (counted as false) and everything else as "truthy". The falsy values are: `false`, `0`, `''`, `null`, `undefined`, `NaN`. Everything else — including any non-empty string, any object, any array (even an empty one!) — is truthy.

```
if (name) { /* runs only if name is a non-empty string */ }
if (list.length) { /* runs only if the array has at least 1 item */ }
```

6. JavaScript Part C — Functions

A function is a named, reusable block of code. You define it once, then **call** (run) it as many times as you like, optionally feeding it different inputs ("arguments" / "parameters") each time.

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(2, 3); // result is 5
```

6.1 Why functions matter here

Tiffin has roughly 100 functions. Breaking a huge program into small, named functions is the single most important organizing idea in this codebase — instead of one giant block of code, there's a function for each distinct job: `saveState()` saves data, `renderToday()` builds the home screen's HTML, `calcBMR()` does one specific calculation, and so on. Each one is small enough to read and understand on its own.

6.2 Return values

A function can hand back a result using `return`. Whatever follows `return` becomes the value of calling that function.

```
function round1(n) {  
  return Math.round((n + Number.EPSILON) * 10) / 10;  
}  
  
round1(3.14159); // returns 3.1
```

This exact function, `round1`, exists in `app.js` and is used everywhere numbers are displayed, to avoid showing ugly values like `3.14159265` to the user.

6.3 Functions as values

In JavaScript, a function is just another kind of value — it can be stored in a variable, put in an array, or passed into another function as an argument. This is used constantly for event handling: "when this button is clicked, run this function."

```
button.addEventListener('click', function() {  
  console.log('clicked!');  
});  
  
// same thing, using a shorter arrow-function syntax (see Chapter 8):  
button.addEventListener('click', () => console.log('clicked!'));
```

7. JavaScript Part D — Arrays & Objects

7.1 Arrays — ordered lists

```
const fruits = ['apple', 'banana', 'cherry'];

fruits[0];           // 'apple' (indexes start at 0!)
fruits.length;       // 3
fruits.push('date'); // adds to the end
```

7.2 Array methods used throughout this app

These four methods appear on almost every page of app.js. Learn them well:

.map() — transform every item into something new

```
const kcals = foods.map(food => food.kcal);
// foods = [{kcal:100}, {kcal:200}] -> kcals = [100, 200]

// Real use: turning a list of foods into a list of HTML strings
const html = foods.map(f => `<div>${f.name}</div>`).join('');
```

.filter() — keep only items that pass a test

```
const cheap = foods.filter(f => f.kcal < 150);
// keeps only foods with fewer than 150 calories
```

.forEach() — do something with every item (no new array returned)

```
NUTRIENT_KEYS.forEach(key => {
  total[key] = (a[key] || 0) + (b[key] || 0);
});
```

.find() — get the first item that matches, or undefined

```
function dbFoodById(id) {
  return FOOD_DB.find(f => f.id === id) || null;
}
```

Note: This exact pattern — `.find(...) || null` — appears constantly in this codebase. `.find()` returns undefined if nothing matches; the `|| null` converts that into an explicit, intentional null, which is easier to check for deliberately later.

.join() — turn an array back into one string

```
['a', 'b', 'c'].join(''); // "abc" — used to combine many HTML snippets into one block
['a', 'b', 'c'].join(' '); // "a, b, c"
```

.reduce() and .sort()

```
// .sort compares two items at a time; return negative to put `a` first
const sorted = [...list].sort((a,b) => a.date < b.date ? -1 : 1);
```

7.3 Objects — labeled bundles of data

```
const food = {
  name: "Rice",
  kcal: 200,
  protein: 4.2
};

food.name;           // "Rice"           – dot notation
food['kcal'];         // 200              – bracket notation (needed for dynamic keys)

const key = 'protein';
food[key];           // 4.2              – you can't do food.key, that would look for a literal "key" property
```

Bracket notation with a variable key is essential in this app because nutrient names are stored as strings in a list (NUTRIENT_KEYS) and looked up dynamically — see Chapter 25.

7.4 Object.keys, Object.assign, and friends

```
Object.keys({a:1, b:2});           // ['a', 'b']
Object.assign({}, obj1, obj2);      // merges obj2's keys into a NEW copy of obj1
Object.entries({a:1, b:2});         // [['a',1], ['b',2]]
```

`Object.assign({}, ...)` — starting from an empty object — is a common way to make a *copy* rather than mutating the original, which avoids accidental side effects.

8. JavaScript Part E — Modern Syntax

8.1 Arrow functions

A shorter way to write a function, especially a short one used as an argument to another function:

```
// Old style
function double(x) { return x * 2; }

// Arrow function, same thing
const double = (x) => { return x * 2; };

// Arrow function, shortest form (implicit return, no braces/return needed)
const double = x => x * 2;

// Used constantly in .map/.filter/.forEach:
foods.map(f => f.kcal);
foods.filter(f => f.category === 'Indian');
```

8.2 Destructuring — unpacking values in one step

```
// Array destructuring
const [first, second] = ['a', 'b']; // first='a', second='b'

// Real example from this app (a nutrient row definition):
const [key, label, unit, goalKey] = ['protein', 'Protein', 'g', 'proteinGoal'];

// Object destructuring
const { name, age } = person; // pulls out person.name and person.age directly
```

8.3 Spread syntax (...) — expanding a collection

```
const a = [1, 2, 3];
const b = [...a, 4, 5]; // [1, 2, 3, 4, 5] – copies `a`'s items, then adds more

const original = {x: 1, y: 2};
const copy = {...original, y: 99}; // {x: 1, y: 99} – copy with one field overridden

// Real example: copying the weight log before sorting, so the original array
// used elsewhere in STATE isn't accidentally reordered:
const sorted = [...STATE.weightLog].sort((a, b) => a.date < b.date ? -1 : 1);
```

8.4 Optional/default parameters and short-circuiting

```
function greet(name) {
  name = name || 'friend'; // if name is falsy (missing), fall back to 'friend'
  return `Hello, ${name}`;
}

opts = opts || {}; // a very common Tiffin pattern: "if no options object was
                  // passed in, use an empty one instead of crashing later"
```


9. JavaScript Part F — The DOM & Events

9.1 What is the DOM?

When a browser loads HTML, it builds a live, in-memory tree of objects representing every element on the page — this tree is called the **DOM** (Document Object Model). JavaScript can read this tree, and — critically — *change* it. Any change to the DOM is instantly reflected on screen; you never have to manually "redraw" anything.

9.2 Finding elements

```
document.getElementById('viewRoot'); // the one element with id="viewRoot"
document.querySelector('.food-row'); // the FIRST element matching this CSS selector
document.querySelectorAll('.food-row'); // ALL elements matching it, as a list
```

9.3 Changing elements

```
el.textContent = 'Hello'; // sets plain text (safe – no HTML parsing)
el.innerHTML = '<b>Hello</b>'; // sets HTML (parses tags – can be dangerous, see Ch.39)
el.classList.add('active'); // adds a CSS class
el.classList.remove('active'); // removes a CSS class
el.style.width = '50%'; // sets one inline CSS property directly
el.setAttribute('data-id', '123'); // sets any HTML attribute
```

Tiffin's entire UI is built by constantly setting `.innerHTML` on a handful of container elements to giant strings of freshly-generated HTML. This is explained fully in Chapter 28.

9.4 Events

An **event** is something that happens — a click, a keystroke, a form submission, the page finishing loading. You **listen** for an event on an element, providing a function to run when it happens:

```
button.addEventListener('click', function(event) {
  console.log('Button was clicked!');
  console.log(event.target); // the exact element that triggered this
});
```

Common events used in this app: `click`, `input` (fires on every keystroke in a text field), `change` (fires when a value is committed, e.g. picking a file), `submit` (fires when a form is submitted), and `DOMContentLoaded` (fires once, when the page has finished loading).

10. JavaScript Part G — Event Delegation, Closures, IIFEs

10.1 Event delegation — the single most important pattern in this app

Imagine a page with 50 "delete" buttons — one per food logged today. You could attach 50 separate click listeners, one to each button. But there's a smarter way, based on a real DOM behavior called **event bubbling**: when you click a button, the click event doesn't just fire on that button — it then "bubbles up" and also fires on its parent, its grandparent, all the way up to document itself.

Event delegation exploits this: instead of 50 listeners, you attach ONE listener to document itself, and when it fires, you check *which element was actually clicked* (via `event.target`) to decide what to do.

```
document.addEventListener('click', function(e) {
  const el = e.target.closest('[data-action]');
  if (!el) return; // click wasn't on anything actionable
  const action = el.dataset.action; // reads the data-action="..." attribute

  if (action === 'delete-entry') { /* ... */ }
  if (action === 'add-water') { /* ... */ }
  // ...dozens more branches
});
```

`closest()` walks up from the clicked element through its ancestors until it finds one with a `data-action` attribute (or gives up and returns null). This means the click could land on an icon INSIDE a button, and it still correctly finds the button.

This is exactly Tiffin's entire strategy: one listener, one big if-chain, ~150 possible actions. New buttons never need new JavaScript to be "wired up" — they just need a `data-action` attribute matching an existing branch. Full detail in Chapter 36.

10.2 Closures

A closure is a function that "remembers" the variables that were in scope where it was defined, even after that outer code has finished running.

```
function makeCounter() {
  let count = 0; // this variable is private to makeCounter
  return function() {
    count = count + 1;
    return count;
  };
}
const counter = makeCounter();
counter(); // 1
counter(); // 2 -- it remembered `count` between calls!
```

10.3 IIFEs — Immediately Invoked Function Expressions

`app.js` is wrapped, top to bottom, in this pattern:

```
(function(){  
  "use strict";  
  // ...the ENTIRE app's code lives in here...  
})();
```

This defines an anonymous function and calls it immediately (note the `()` at the very end). Why bother? Because every `const/let` declared inside is scoped to that function — completely invisible to the outside world. This keeps the app's ~100 internal function and variable names from ever clashing with anything else on the page (like the Chart.js library it loads separately). It's a lightweight, no-tooling substitute for what other projects achieve with a module bundler.

"use strict" turns on JavaScript's stricter rule set — for example, it makes accidentally creating a global variable (by forgetting `const/let`) throw an error instead of silently succeeding.

11. JavaScript Part H — Async Code & Promises

Some operations take time and shouldn't freeze the whole page while waiting: asking for camera access, reading a file the user picked, fetching something from the network. JavaScript handles this with **Promises** and the `async/await` keywords.

```
async function startCamera() {  
  // `await` pauses THIS function (not the whole page) until the promise resolves  
  const stream = await navigator.mediaDevices.getUserMedia({ video: true });  
  video.srcObject = stream;  
}
```

A function marked `async` can use `await` inside it. `await somePromise` means "pause here, let other things on the page keep working, and resume this function once the promise finishes." Tiffin uses this for camera access (barcode scanning, Chapter 34) and for its optional "save file" integration (Chapter 35).

```
try {  
  const stream = await navigator.mediaDevices.getUserMedia({video:true});  
} catch (err) {  
  // runs if the user denies camera permission, or no camera exists  
  showMessage("Camera access isn't available here.");  
}
```

`try { ... } catch (err) { ... }` is how JavaScript handles errors: code that might fail goes in the `try` block; if it throws an error, execution jumps straight to `catch` instead of crashing the whole app.

12. JSON — The Data Format

JSON (JavaScript Object Notation) is a text format for representing data — arrays, objects, strings, numbers, booleans, null. It looks almost exactly like JavaScript object/array syntax, with a few stricter rules (keys must be in double quotes; no comments allowed; no trailing commas).

```
{
  "id": "rice-cooked-white",
  "name": "Rice, cooked (Chawal, white)",
  "category": "Indian",
  "kcal": 200,
  "protein": 4.2,
  "tags": ["staple", "gluten-free"]
}
```

JSON is the universal language for storing and transmitting structured data. In this project it's used for three separate things:

- **food_db.json** — the entire 332-food nutrition database (Chapter 22).
- **localStorage** — the user's own data (profile, diary, custom foods) is converted to a JSON string to be saved in the browser, then parsed back when the app reopens (Chapter 24).
- **package.json / capacitor.config.json** — configuration files that describe the project to various tools (Node.js, Capacitor).

Converting between JavaScript objects and JSON text

```
const data = { name: "Alice", age: 30 };

const text = JSON.stringify(data);    // '{"name":"Alice","age":30}' (a STRING)
const back = JSON.parse(text);        // { name: "Alice", age: 30 } (an OBJECT again)
```

`JSON.stringify` turns a JS value into JSON text (used when saving); `JSON.parse` turns JSON text back into a live JS value (used when loading). If the text isn't valid JSON, `JSON.parse` throws an error — which is exactly why it's always wrapped in a `try/catch` in this app.

13. Browser Storage — localStorage

Normally, anything a JavaScript variable holds disappears the instant the tab is closed. `localStorage` is a small key-value database built into every browser that *persists* — it survives page refreshes, tab closes, even computer restarts. It's tied to one website's origin (domain), and typically allows a few megabytes of storage — plenty for this app's data.

```
localStorage.setItem('myKey', 'some text value'); // save (must be a string!)
const value = localStorage.getItem('myKey');      // load (or null if never set)
localStorage.removeItem('myKey');                 // delete one key
localStorage.clear();                             // delete everything
```

Because `localStorage` only stores *strings*, saving a whole object means combining it with JSON (Chapter 12) — stringify before saving, parse after loading. This exact pattern is the entire persistence layer of Tiffin, covered in full in Chapter 24:

```
localStorage.setItem('tiffin_state_v1', JSON.stringify(STATE));

const raw = localStorage.getItem('tiffin_state_v1');
const STATE = JSON.parse(raw);
```

Note: This is why Tiffin needs no backend server, no account, no login. Your data lives only on your own device, inside your browser. The tradeoff: it doesn't sync between devices, and clearing your browser data would erase it — which is exactly why the app has an Export/Import backup feature (Chapter 35).

14. Browser APIs — Camera, Files, Clipboard

Beyond the DOM, browsers expose many built-in "Web APIs" that JavaScript can call to do more advanced things. Tiffin uses a few:

BarcodeDetector

A relatively new browser API that can scan a video feed for barcodes and report what it finds, with no external library needed. Not every browser supports it, so Tiffin always checks first: `if ('BarcodeDetector' in window)` before trying to use it, and falls back to a manual numeric-entry field if it's unavailable.

MediaDevices / getUserMedia

Requests access to the device's camera (or microphone). Returns a live video "stream" that can be shown inside a `<video>` tag. Always requires the user's explicit permission.

FileReader

Reads the contents of a file the user has selected (e.g. via a file-picker input), asynchronously, so large files don't freeze the page. Used for the "Import backup" feature.

```
const reader = new FileReader();
reader.onload = () => {
  const text = reader.result; // the file's full text content
  const data = JSON.parse(text);
};
reader.readAsText(file);
```

Clipboard API

`navigator.clipboard.writeText(...)` copies text to the system clipboard programmatically — used by the "Copy to clipboard" button on the export screen.

Chart.js (a third-party library, not a browser API)

Tiffin doesn't reinvent chart-drawing. It loads the popular open-source Chart.js library dynamically from a CDN (Content Delivery Network — a server that just hosts popular public files) only when the user opens the Progress tab, to avoid slowing down initial page load for people who never look at it. Details in Chapter 32.

15. Python — Just Enough to Read build.py

Python is a different programming language from JavaScript, used here for exactly one job: a small build script that glues the app's files together before shipping (Chapter 19). You don't need deep Python knowledge — just enough to follow ~85 lines of fairly readable code.

15.1 What's different from JavaScript (at a glance)

Concept	JavaScript	Python
Variable	<code>const x = 5;</code>	<code>x = 5</code>
Blocks	<code>{ curly braces }</code>	indentation (whitespace) defines blocks
String	<code>"text" or `\${x}`</code>	<code>'text' or f"{x}"</code>
Function	<code>function f(x) { return x; }</code>	<code>def f(x): return x</code>
Import code	<code><script src=...></code>	<code>import module_name</code>

15.2 Reading and writing files

```
import pathlib

path = pathlib.Path("app.js")
text = path.read_text(encoding="utf-8") # read the whole file as a string
path.write_text("new content", encoding="utf-8") # overwrite a file
```

15.3 f-strings — Python's template literals

```
name = "world"
print(f"Hello, {name}!") # "Hello, world!" -- just like JS's `Hello, ${name}!`
```

15.4 String methods used in build.py

```
text.replace("__CSP__", actual_csp_value) # find-and-replace, like JS's .replace()
```

build.py uses this repeatedly to substitute placeholders like `/*__CSS__*/` inside `shell.html` with real file contents — see Chapter 19.

16. Git & GitHub Actions — Version Control and CI/CD

16.1 Git — tracking changes over time

Git is a **version control system**: it records a history of every change made to a project's files, letting you see what changed, when, and by whom — and revert if something breaks.

- A **commit** is a saved snapshot of the project at one point in time, with a message describing what changed.
- A **branch** is an independent line of development — you can experiment on a branch without touching the main, working version.
- **Pushing** uploads your local commits to a remote copy (usually on GitHub); **pulling** downloads new commits from there.

16.2 GitHub Actions — automation that runs on every push

GitHub Actions runs scripts automatically in response to events in a repository — most commonly, "whenever someone pushes new code." These automated scripts are called **workflows**, and this practice — automatically building and testing code the moment it changes — is called **CI/CD** (Continuous Integration / Continuous Deployment).

This project has two workflow files, each a small YAML (a simple, indentation-based configuration format) recipe of steps:

```
# .github/workflows/build-android.yml (simplified)
on:
  push:
    branches: [ main, claude/friendly-davinci-kz02xp ]

jobs:
  build:
    runs-on: ubuntu-latest      # a fresh Linux machine, just for this job
    steps:
      - uses: actions/checkout@v4      # download the repo's code
      - run: npm install                # install dependencies
      - run: npm run build              # run the Python build script
      - run: npx cap sync android       # copy the built web app into the Android project
      - run: cd android && ./gradlew assembleRelease # compile the Android app
      - uses: actions/upload-artifact@v4 # make the finished .apk downloadable
```

Every time code is pushed to this project's branch, GitHub spins up two temporary virtual machines (one for Android, one for iOS/macOS), each runs its recipe end to end, and the finished app files (.apk for Android, .ipa for iOS) become downloadable "artifacts" — with zero manual work.

17. Capacitor — Turning a Website into a Phone App

Capacitor is a tool that takes a normal web app (HTML/CSS/JS) and wraps it inside a real native mobile app shell, for both iOS and Android. Concretely, it creates a native app whose entire screen is just a full-screen embedded browser (called a "WebView") pointed at your built HTML file. Your JavaScript keeps running exactly as it does in a normal browser tab — Capacitor's main value is packaging it as something installable from an app store, with access to native device features if needed.

```
// capacitor.config.json
{
  "appId": "com.example.tiffin",
  "appName": "Tiffin",
  "webDir": "dist"    // <-- tells Capacitor which folder holds the built web app
}
```

The workflow is: 1) build.py assembles the single dist/index.html file, 2) `npx cap sync android` (or `ios`) copies that file into a native Android Studio / Xcode project template that Capacitor generated once, 3) the platform's own native build tool (Gradle for Android, Xcode for iOS) compiles that into an actual installable app. This is exactly what the two GitHub Actions workflows from Chapter 16 automate.

Note: Because Tiffin already stores everything in `localStorage` and needs no server, wrapping it with Capacitor required almost no extra code — the same JavaScript that runs in a desktop browser tab runs unmodified inside the native app's embedded `WebView`.

PART 2

The App, Explained End to End

With that vocabulary in hand, we can now read every file in this project and understand exactly what it does and why. We'll go file by file, then work through `app.js` — the largest and most important file — section by section, in the order it's actually written.

18. Project Tour — Every File and What It's For

File	Language	Job
shell.html	HTML	The empty page skeleton — a few containers JS fills in
app.css	CSS	All visual styling, including dark mode and responsive layout
app.js	JavaScript	The entire application logic — ~1,700 lines, one big IIFE
food_db.json	JSON	332 pre-loaded foods with full nutrition data
build.py	Python	Glues the four files above into one dist/index.html
package.json	JSON	Project metadata + npm scripts + dependency list
capacitor.config.json	JSON	Tells Capacitor which folder is the built web app
.github/workflows/*.yaml	YAML	CI/CD recipes that auto-build Android/iOS apps

Notice what's *not* here: no backend server code, no database server, no API. Everything the app needs — the food database included — ships as static files, and all logic runs inside the user's own browser. This single decision shapes nearly everything else about the project (Chapter 38).

19. The Build Pipeline — build.py Deep Dive

shell.html contains four placeholder markers that aren't real HTML — they're just text build.py knows to search and replace:

```
<style>
/*__CSS__*/
</style>
...
<script>
const FOOD_DB = /*__FOODDB__*/;
</script>
<script>
/*__APPJS__*/
</script>
```

build.py reads all four source files, then does a simple find-and-replace for each placeholder, producing one giant, fully self-contained HTML file at dist/index.html — CSS, food data, and app logic all inlined directly into the page, no separate file requests needed:

```
shell = (ROOT / "shell.html").read_text(encoding="utf-8")
css = (ROOT / "app.css").read_text(encoding="utf-8")
appjs = (ROOT / "app.js").read_text(encoding="utf-8")
fooddb = (ROOT / "food_db.json").read_text(encoding="utf-8")

out = (
    shell.replace("__CSP__", csp)
        .replace("/*__CSS__*/", css)
        .replace("/*__FOODDB__*/", fooddb)
        .replace("/*__APPJS__*/", appjs)
)
output_path.write_text(out, encoding="utf-8")
```

Why bother splitting the source into 4 files just to glue them back together? **Developer ergonomics.** Editing app.js in a dedicated .js file gets you syntax highlighting, linting, and autocomplete in a code editor — none of which work well for JS embedded inside an HTML string. Splitting the source and reassembling it at build time gets the best of both: a nice editing experience, and a single zero-dependency HTML file to actually ship.

19.1 The Content-Security-Policy (CSP) — a security feature

The trickiest part of build.py computes a cryptographic **SHA-256 hash** (a unique fingerprint) of the exact text inside each of the two inline <script> tags:

```
def sha256_b64(text):
    return base64.b64encode(hashlib.sha256(text.encode("utf-8")).digest()).decode("ascii")

script_hashes = " ".join(
    f"sha256-{sha256_b64(body)}" for body in (fooddb_script_body, appjs_script_body)
)
```

This hash gets embedded into a Content-Security-Policy HTTP header/meta tag that tells the browser: "only run inline scripts whose exact content matches one of these fingerprints — refuse to run

anything else." If an attacker somehow injected a rogue `<script>` tag into the page (Chapter 39 explains how that could even happen), its hash wouldn't match, and the browser would simply refuse to execute it. This is a strong, defense-in-depth security measure explained fully in Chapter 39.

20. shell.html Deep Dive

Once you strip out the placeholders, shell.html is almost nothing — deliberately so:

```
<div class="topbar">...</div>                                <!-- mobile-only top bar -->
<div class="app">
  <nav class="sidenav" id="sideNav"></nav>                    <!-- EMPTY – JS fills this -->
  <main class="main">
    <div id="viewRoot"></div>                                <!-- EMPTY – JS fills this -->
  </main>
</div>
<nav class="bottomnav" id="bottomNav"></nav>                  <!-- EMPTY – JS fills this -->
<div class="modal-overlay" id="modalOverlay">
  <div class="modal" id="modalContent"></div>                  <!-- EMPTY – JS fills this -->
</div>
<div class="toast-wrap" id="toastWrap"></div>                 <!-- EMPTY – JS fills this -->
```

Five containers, each with an `id`, and every single one starts completely empty. This is the hallmark of a JavaScript-driven single page app: HTML provides only the *slots*; JavaScript generates 100% of what actually appears inside them, and keeps regenerating it as the app's data changes. We'll see exactly how in Chapter 28.

21. app.css Deep Dive

21.1 Design tokens and dark mode, revisited

Recall from Chapter 3 that CSS variables let you name a value once and reuse it everywhere. `app.css` defines its full palette under `:root`, then *redefines the exact same variable names* inside a dark-mode block:

```
:root {
  --paper: #FBF3E7; /* warm cream background */
  --text: #241C12; /* near-black text */
  --accent: #FF6B4A; /* the brand orange */
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --paper: #161210; /* near-black background */
    --text: #F5ECDF; /* warm off-white text */
    --accent: #FF7D5E; /* a slightly brighter orange, for contrast on dark */
  }
}

:root[data-theme="dark"] {
  /* the SAME dark values, but applied whenever the app's own theme setting
   is explicitly "dark" — regardless of the OS setting */
  --paper: #161210; --text: #F5ECDF; --accent: #FF7D5E;
}
```

Three theme states are supported: **system** (follow the OS's light/dark preference automatically, via the `@media (prefers-color-scheme: dark)` query), or an explicit **light** / **dark** override, applied by JavaScript setting a `data-theme` attribute on the `<html>` element (Chapter 35 shows the JS side of this). Not one other CSS rule in the whole file needs to know or care which theme is active — they all just say `color: var(--text)`, and the variable's current value does the rest.

21.2 Mobile vs desktop layout

Recall the media query pattern from Chapter 3.6. `app.css` uses exactly one breakpoint, 860px, to switch between a desktop sidebar and a mobile bottom tab bar:

```
.sidenav { width: 216px; /* ...desktop sidebar styles... */ }
.bottomnav { display: none; /* hidden by default (desktop) */ }

@media (max-width: 860px) {
  .sidenav { display: none; } /* hide sidebar on narrow screens */
  .bottomnav { display: flex; } /* show the tab bar instead */
  .topbar { display: flex; } /* show the mobile top bar */
}
```

Both navigation elements exist in the HTML at all times — CSS alone decides which one is visible, based purely on the browser window's width, no JavaScript involved.

21.3 Reduced motion accessibility

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    animation-duration: .001ms !important;  
    transition-duration: .001ms !important;  
  }  
}
```

Some users configure their OS to minimize animations (often for motion-sensitivity/vestibular reasons). This query respects that system preference by effectively disabling all CSS animations and transitions for them, app-wide, in one rule.

22. food_db.json Deep Dive

This file is a single JSON array containing 332 food objects. Every object has the exact same set of keys — the same 44 nutrient fields, always present (even if zero), plus a few descriptive fields:

```
{
  "id": "rice-cooked-white",
  "name": "Rice, cooked (Chawal, white)",
  "category": "Indian",
  "servingLabel": "1 cup",
  "servingGrams": 150,
  "kcal": 200,
  "protein": 4.2,
  "carbs": 44,
  "fat": 0.4,
  "satFat": 0.1,
  "fiber": 0.6,
  ... (continues through all 44 nutrients – vitamins, minerals, fats, etc.)
}
```

- `id` — a unique, human-readable identifier (used to reference this food from the diary, never the display name, since names could change).
- `category` — 'Indian' or 'International', used for the filter chips in the Foods tab.
- `servingLabel` / `servingGrams` — a human-friendly serving description ("1 cup") paired with its weight in grams, so any other serving size can be computed proportionally.
- Every other key matches one entry in the app's own `NUTRIENT_KEYS` list (Chapter 23) — this consistency is what lets the same generic nutrient-math functions work identically on every food.

Note: This file is loaded once, embedded directly as a JS array literal via `build.py`'s `__FOODDB__` placeholder (Chapter 19) — there's no network request to fetch it at runtime, which is why the app works completely offline.

23. app.js Part 1 — Setup & The Shape of the Data

Everything from here through Chapter 37 walks through app.js top to bottom. Line numbers refer to the actual file in this repository.

23.1 The wrapper (lines 1-8)

```
(function(){  
  "use strict";
```

As explained in Chapter 10.3, this is an IIFE: it isolates every name the app defines from the rest of the page, and turns on JavaScript's strict mode.

23.2 Icons (lines 9-27)

An object literal, `ICON`, mapping short names like `home`, `flame`, `x` to raw SVG markup strings. Recall from Chapter 2 that `<svg>` draws vector graphics directly in markup — no image files are downloaded for any icon in this app; they're all just text, embedded directly into whatever HTML string needs them, e.g. `${ICON.flame}`.

23.3 Constants (lines 29-114)

```
const STORAGE_KEY = 'tiffin_state_v1'; // the localStorage key everything is saved under  
  
const NUTRIENT_KEYS = [  
  'kcal', 'protein', 'carbs', 'fat', 'satFat', 'fiber', 'sugar', 'sodium', ... // all 44, by name  
];
```

`NUTRIENT_KEYS` is the single source of truth for "what nutrients does this app track?" — every nutrient-math function (Chapter 25) loops over this list rather than hardcoding 44 separate lines, so adding a 45th nutrient in the future would mean editing this one list, not every function in the file.

```
const NUTRIENT_GROUPS = [  
  {title: 'Macros', rows: [  
    ['protein', 'Protein', 'g', 'proteinGoal'],  
    ['carbs', 'Carbohydrates', 'g', 'carbGoal'],  
    ['fat', 'Fat', 'g', 'fatGoal'],  
    ...  
  ]},  
  {title: 'Vitamins', rows: [ ... ]},  
  {title: 'Minerals', rows: [ ... ]},  
  ...  
];
```

`NUTRIENT_GROUPS` is a richer structure used purely for display: it groups related nutrients under headings (Macros, Vitamins, Minerals...) and, per nutrient, bundles its internal key, its human-readable label, its unit, and which goal setting (if any) it's measured against. This one list drives both the Nutrients tab (Chapter 27) and the custom-food-entry form (Chapter 29) — define a nutrient's metadata once, get two different screens generated from it automatically.

```

const FIXED_TARGETS = {
  addedSugarLimit: 50, omega3AlaGoal: 1.4,
  vitB1Goal: 1.2, vitB2Goal: 1.3, ... // general adult nutrition reference values
};

const ACTIVITY_MULT = {sedentary:1.2, light:1.375, moderate:1.55, active:1.725, very_active:1.9};

const DEFAULT_GOALS = {
  calorieGoal: 2000, proteinGoal: 100, carbGoal: 250, fatGoal: 65, ...
};

```

FIXED_TARGETS holds general recommended daily amounts (for nutrients too obscure to let users customize) — merged in at display time, never stored per-user. ACTIVITY_MULT holds the standard multipliers used in the Mifflin-St Jeor energy formula (Chapter 26). DEFAULT_GOALS is what a brand-new user starts with before they customize anything in Profile.

23.4 Default-state factory functions (lines 116-133)

```

function defaultMealsSlots(){
  return [
    {id:'meal-1', name:'Meal 1', emoji:'■'},
    {id:'meal-2', name:'Meal 2', emoji:'■'},
    ... // 5 meals by default
  ];
}
function defaultProfile(){
  return { name:'', sex:null, age:null, heightCm:null, ... };
}
function defaultState(){
  return {
    goals: {...DEFAULT_GOALS}, profile: defaultProfile(), diary: {}, weightLog: [],
    customFoods: [], customMeals: [], barcodeMap: {}, mealSlots: defaultMealsSlots(),
  };
}

```

Notice each of these is a *function that returns a fresh object*, not a shared constant object. This matters: if `defaultState()` were instead one constant object reused everywhere, every new user (and the "Reset all data" feature, Chapter 35) would end up sharing and accidentally mutating the exact same object in memory. Calling a function guarantees a brand new, independent copy every time.

This block also defines the complete shape of the app's data: goals (calorie/macro targets), profile (the user's body stats), diary (a dictionary of date → meals eaten), weightLog (an array of weigh-ins), customFoods / customMeals (anything the user created themselves), barcodeMap (barcode numbers linked to specific foods), and mealSlots (the user's customizable list of meal names, replacing a fixed "Breakfast/Lunch/Dinner" scheme).

24. app.js Part 2 — Storage & Security

24.1 Loading and saving (lines 150-190)

```
let STATE = loadState(); // <-- the app's ENTIRE data lives in this one variable

function saveState(){
  try{ localStorage.setItem(STORAGE_KEY, JSON.stringify(STATE)); }
  catch(e){ storageOk = false; }
}
```

This is the pattern from Chapter 13, applied to the whole app's data at once. Every single place in app.js that changes something — adding a food, logging weight, changing a setting — calls `saveState()` immediately afterward. There is no "Save" button for the app as a whole; changes are persisted the instant they happen.

The try/catch matters because `localStorage` can fail — private/incognito browsing sometimes disables it entirely, or storage could be full. Rather than crash, the app sets a flag (`storageOk = false`) and later warns the user their data won't be saved, so they know to back it up manually (Chapter 35).

24.2 mergeKnownFields — a security-conscious loader

This is the single most important function to understand for interview purposes, because it fixes a real, named security vulnerability class. Read the code and its own comment carefully:

```
// Copies only keys already present on `defaults` from `source`, so a
// stored/imported JSON blob can never introduce arbitrary keys (including
// "__proto__", which Object.assign(target, source) would otherwise use to
// repoint target's prototype -- CWE-1321 prototype pollution).
function mergeKnownFields(defaults, source){
  const out = Object.assign({}, defaults);
  if(source && typeof source === 'object'){
    Object.keys(defaults).forEach(k=>{
      if(Object.prototype.hasOwnProperty.call(source, k)) out[k] = source[k];
    });
  }
  return out;
}
```

This deserves the full explanation, because it's exactly the kind of thing an interviewer will probe. The naive way to merge "saved settings" over "default settings" would be:

```
const merged = Object.assign({}, defaults, source); // DANGEROUS with untrusted `source`
```

The problem: every JavaScript object secretly has a hidden `__proto__` property pointing to its prototype (the object it inherits shared behavior from). If `source` came from untrusted JSON — like an imported backup file someone could hand-edit, or in principle data planted by an attacker — and that JSON contained a key literally named `"__proto__"`, a naive `Object.assign` would follow it and let the attacker inject properties onto `Object.prototype` itself, corrupting the behavior of every object in the entire running app. This class of bug has an official name and ID: **CWE-1321, Prototype Pollution**.

`mergeKnownFields` defends against this in two ways at once: (1) it only ever copies keys that already exist on `defaults` — since `defaults` is always a hardcoded object this app itself wrote, `"__proto__"` is never one of its keys, so the loop simply never looks at it; and (2) it uses `Object.prototype.hasOwnProperty.call(source, k)` rather than the simpler `source.hasOwnProperty(k)` or `k in source` — the former can itself be spoofed if `source` has been tampered with to no longer inherit the normal `Object` prototype; calling the trusted, original method explicitly sidesteps that.

This function is used in two places: loading state from `localStorage` on startup (which, strictly, is data the app itself wrote, so lower risk) and importing a backup file the user selects (Chapter 35) — the more realistic risk, since backup files could be shared, edited, or come from an untrusted source.

24.3 Legacy data migration

`loadState()` also contains a one-time migration for users who had the app before "meal slots" became customizable, converting their old fixed `breakfast/lunch/dinner/snacks` keys into the new generic `meal-1/meal-2/...` scheme automatically, so nobody's existing diary history is lost when the app is updated.

25. app.js Part 3 — Utility & Math Functions

25.1 General-purpose helpers (lines 193-210)

```
function uid(){ return Math.random().toString(36).slice(2,9); }
function round1(n){ return Math.round((n + Number.EPSILON) * 10) / 10; }
function clamp(v,a,b){ return Math.max(a, Math.min(b, v)); }
function escapeHtml(s){ return String(s).replace(/[\&<>"]/g, c=>({...}[c])); }
```

- `uid()` generates a short random ID — used for every custom food, custom meal, and meal slot the user creates, so each can be referenced uniquely.
- `round1()` rounds to 1 decimal place, used for nearly every displayed number.
- `clamp(v, a, b)` forces a value to stay between `a` and `b` (used to keep progress bars from visually overflowing past 100%).
- `escapeHtml()` is Tiffin's central XSS defense — full explanation in Chapter 39.

25.2 Date helpers

```
function toKey(d){ return d.getFullYear()+'-'+pad2(d.getMonth()+1)+'-'+pad2(d.getDate()); }
function todayKey(){ return toKey(new Date()); }
function addDaysKey(key, delta){ const d = keyToDate(key); d.setDate(d.getDate()+delta); return toKey(d); }
```

The whole app represents a date not as a JavaScript Date object, but as a plain string key like `'2024-09-17'`. This makes it trivially usable as an object key (for the diary, see Chapter 26) and easy to compare/sort as plain strings, while still allowing arithmetic ("give me yesterday") by briefly converting back to a real Date object, changing it, and converting back to a key.

25.3 Nutrient math — the core arithmetic of the whole app

```
function zeroNutrients(){
  const o = {}; NUTRIENT_KEYS.forEach(k=>o[k]=0); return o;
}
function scaleNutrients(base, qty){
  const o = {}; NUTRIENT_KEYS.forEach(k=> o[k] = (base[k]||0) * qty); return o;
}
function addNutrients(a,b){
  const o = {}; NUTRIENT_KEYS.forEach(k=> o[k] = (a[k]||0) + (b[k]||0)); return o;
}
```

These three tiny functions are the mathematical foundation everything else builds on, and they're a perfect example of the `NUTRIENT_KEYS` list paying off (Chapter 23.3): instead of 44 lines of `o.protein = a.protein + b.protein; o.carbs = a.carbs + b.carbs; ...`, one loop handles every current and future nutrient identically.

- `zeroNutrients()` — an empty starting point (everything 0), used as the running total before summing a day's meals.
- `scaleNutrients(base, qty)` — multiplies every nutrient by a quantity, e.g. "1.5 servings of rice" scales rice's per-serving nutrients by 1.5.

- `addNutrients(a, b)` — adds two nutrient sets together, key by key, e.g. combining breakfast's total with lunch's total.

25.4 Resolving a "reference" into actual nutrition data

A logged diary entry doesn't store a copy of a food's nutrients — it stores a lightweight *reference*: `{type: 'db', id: 'rice-cooked-white'}` plus how many servings. This keeps the diary small and means editing a custom food's nutrition later automatically updates every past log entry that referenced it.

```
function resolveRefBase(ref){
  if(!ref) return null;
  if(ref.type === 'db'){
    const f = dbFoodById(ref.id); if(!f) return null;
    return {name:f.name, servingLabel:f.servingLabel, servingGrams:f.servingGrams, nutrients:f, ...};
  }
  if(ref.type === 'custom'){ ... }
  if(ref.type === 'meal'){
    // a custom "meal" (recipe) is a LIST of other refs -- resolve each ingredient
    // recursively and sum them together
    let n = zeroNutrients(); let grams = 0;
    m.items.forEach(it=>{
      const b = resolveRefBase(it.ref);
      if(!b) return;
      n = addNutrients(n, scaleNutrients(b.nutrients, it.qty));
      grams += (b.servingGrams||0) * it.qty;
    });
    return {name:m.name, servingLabel:'1 recipe', servingGrams:grams, nutrients:n, ...};
  }
  return null;
}
```

There are exactly three kinds of reference: 'db' (one of the 332 built-in foods), 'custom' (a food the user created), and 'meal' (a custom recipe made of several other foods). The 'meal' case is recursive — it calls `resolveRefBase` again on each of its own ingredients — which correctly handles a recipe containing several ingredients, each individually looked up and summed, in one small function.

26. app.js Part 4 — The Diary & Meal System

Recall from Chapter 23 that `STATE.diary` is an object mapping date-key strings to that day's data:

```
STATE.diary = {
  '2024-09-17': {
    'meal-1': [ {ref:{type:'db', id:'rice-cooked-white'}, qty: 1.5}, ... ],
    'meal-2': [ ... ],
    water: 2000,           // ml logged
    workouts: [ {name:'Run', duration:30} ]
  },
  '2024-09-16': { ... }
};
```

```
function getDay(key){
  if(!STATE.diary[key]) STATE.diary[key] = {};
  const d = STATE.diary[key];
  STATE.mealSlots.forEach(s=>{ if(!d[s.id]) d[s.id] = []; });
  if(typeof d.water !== 'number') d.water = 0;
  if(!Array.isArray(d.workouts)) d.workouts = [];
  return d;
}
```

`getDay()` is a "get-or-create" function — a very common pattern. Rather than every caller checking "does today's entry exist yet?", this one function guarantees a fully-formed day object always comes back, creating and filling in any missing pieces (each meal slot's empty array, a zero water total, an empty workout list) on the fly. This also means adding a brand new meal slot later still works for old diary days that were created before that slot existed — `getDay()` patches it in automatically.

```
function dayTotals(key){
  const d = getDay(key);
  let total = zeroNutrients();
  STATE.mealSlots.forEach(s=>{
    (d[s.id]||[]).forEach(entry=>{ total = addNutrients(total, entryNutrients(entry)); });
  });
  return total;
}
```

`dayTotals()` combines everything from Chapter 25: for every meal slot, for every entry logged in it, resolve its nutrients (scaled by quantity) and fold it into a running total with `addNutrients`. This one function powers the calorie ring, the macro bars, and the entire `Nutrients` tab.

```
function computeStreak(){
  let d = todayKey();
  if(!dayHasEntries(d)) d = addDaysKey(d,-1); // don't break the streak just because
                                              // today hasn't been logged YET

  let count = 0;
  while(dayHasEntries(d)){ count++; d = addDaysKey(d,-1); }
  return count;
}
```

The "logging streak" (shown as a flame icon + number) counts consecutive days backward from today that have at least one food logged. The small but thoughtful detail: if *today* has nothing logged yet, it starts

counting from yesterday instead — so your streak doesn't visually reset to zero first thing in the morning before you've had a chance to log breakfast.

27. app.js Part 5 — Energy Calculations (BMR/TDEE)

This section implements real nutrition science: the **Mifflin-St Jeor equation**, a widely-used formula for estimating how many calories someone burns per day at rest.

```
function calcBMR(){
  const p = STATE.profile;
  if(!p.sex || !p.age || !p.heightCm) return null; // can't calculate without these
  const weight = latestWeightKg();
  if(weight == null) return null;
  const s = p.sex === 'male' ? 5 : -161;
  return 10*weight + 6.25*p.heightCm - 5*p.age + s;
}
```

BMR (Basal Metabolic Rate) is roughly "calories burned just to stay alive, lying still all day." The formula needs weight (kg), height (cm), age (years), and biological sex (the sole reason this formula needs it: average differences in body composition), each multiplied by its own scientifically-derived coefficient. Note the guard clauses at the top: if any required profile field is missing, the function returns `null` rather than calculating with garbage/missing data — and every caller of this function is expected to check for that `null` before using the result (Chapter 31 shows the UI's fallback message when it's missing).

```
function calcTDEE(){
  const bmr = calcBMR(); if(bmr == null) return null;
  const mult = ACTIVITY_MULT[STATE.profile.activityLevel || 'moderate'];
  return bmr * mult;
}
```

TDEE (Total Daily Energy Expenditure) scales BMR up by an activity multiplier (from Chapter 23.3's `ACTIVITY_MULT`) to account for exercise and daily movement — someone who's very active burns meaningfully more than their BMR alone.

```
function calculateTargetsFromProfile(){
  const bmr = calcBMR();
  if(bmr == null){ toast('Add your sex, age, and height in Profile first.');
```

`return; }`

```
  const weight = latestWeightKg();
  const tdee = calcTDEE();
  const p = STATE.profile;
  const dailyAdj = ((p.weeklyRateKg||0) * 7700) / 7; // 7700 kcal ~ 1 kg of body fat
  let calorieGoal = tdee;
  if(p.goalType === 'cut') calorieGoal = tdee - dailyAdj;
  else if(p.goalType === 'bulk') calorieGoal = tdee + dailyAdj;
  calorieGoal = Math.max(1200, Math.round(calorieGoal)); // never recommend below 1200 kcal
  const proteinPerKg = p.goalType === 'cut' ? 2.0 : 1.8;
  const proteinGoal = Math.round(weight * proteinPerKg);
  const fatGoal = Math.round(calorieGoal*0.25/9); // ~25% of calories from fat (9 kcal/g)
  const carbGoal = Math.max(0, Math.round((calorieGoal - proteinGoal*4 - fatGoal*9)/4)); // remainder as carb
  ...
}
```

This function turns the science into an actionable set of daily goals with one button click ("Calculate my targets", Chapter 31). It uses the well-known approximation that roughly 7,700 kcal of sustained surplus or deficit corresponds to about 1 kg of body weight change, to translate a user's desired weekly rate of loss/gain into a daily calorie adjustment. It sets protein higher (per kg of bodyweight) during a "cut" (calorie

deficit) than during maintenance or a "bulk," reflecting real nutrition guidance to preserve muscle while losing fat. Fat and carbs are then derived from what's left of the calorie budget, using the standard calorie-per-gram figures (9 kcal/g fat, 4 kcal/g carbs/protein). Note the safety floor: `Math.max(1200, . . .)` refuses to ever suggest a dangerously low calorie target, regardless of what the math alone would produce.

28. app.js Part 6 — The Rendering System

This is the architectural core of the whole UI, and it's worth understanding deeply, because it's the pattern that replaces what a framework like React would normally provide.

```
function renderApp(){ renderShell(); renderView(); }

function renderShell(){
  document.getElementById('sideNav').innerHTML = `...`; // nav, streak, theme buttons
  document.getElementById('bottomNav').innerHTML = renderNav();
  document.getElementById('topbarTitle').textContent = ({...})[currentView];
}

function renderView(){
  const root = document.getElementById('viewRoot');
  let html = '';
  if(currentView === 'today') html = renderToday();
  else if(currentView === 'foods') html = renderFoodsView();
  else if(currentView === 'progress') html = renderProgressView();
  else if(currentView === 'nutrients') html = renderNutrientsView();
  else if(currentView === 'profile') html = renderProfileView();
  root.innerHTML = `<div class="fade-in">${html}</div>`;
}
```

The pattern, in one sentence: **every render function is a pure function that takes the current STATE and returns an HTML string** — it reads data, builds a template-literal string full of `${...}` interpolations, and hands it back. Nothing inside `renderToday()`, for instance, ever touches the DOM directly; it just returns text. The *caller* — `renderView()` — is the only place that actually assigns to `.innerHTML`, replacing everything inside `#viewRoot` in one shot.

28.1 The render cycle — how a click becomes a screen update

This five-step loop happens every single time the user does anything at all in this app:

- **1.** User clicks/types/submits something.
- **2.** The event delegation system (Chapter 36) figures out what action that corresponds to.
- **3.** The handler code directly mutates STATE (e.g. pushes a new entry into today's meal array).
- **4.** The handler calls `saveState()` (persists to `localStorage`) and then `renderView()` or `renderApp()`.
- **5.** The relevant render function reads the now-updated STATE and regenerates the HTML from scratch; the browser updates the screen.

Note: This is a much simpler, hand-rolled version of the same idea frameworks like React are built around: "the UI is a function of state." The tradeoff for not using a real framework is efficiency — Tiffin regenerates and replaces entire chunks of HTML on every change (no fine-grained diffing), which is wasteful in theory, but for an app this size, running on a modern device, it's fast enough to be invisible to the user, and it comes with zero framework dependency and near-zero build tooling. This is a real, defensible trade-off — see Chapter 38.

28.2 Lazy-loading Chart.js

```
function ensureChartLoaded(cb){
  if(typeof Chart !== 'undefined'){ cb(); return; }           // already loaded, run callback now
  if(chartLoadState === 'loading'){ /* wait and retry */ return; }
  chartLoadState = 'loading';
  const s = document.createElement('script');
  s.src = CHART_CDN_URL;
  s.onload = ()=>{ chartLoadState='loaded'; cb(); };
  s.onerror = ()=>{ chartLoadState='error'; cb(); };
  document.head.appendChild(s);
}
```

Rather than always loading the (fairly large) Chart.js library up front, the app only fetches it the first time the user opens the Progress tab, by dynamically creating a `<script>` element pointed at a CDN URL and inserting it into the page. It tracks loading state so simultaneous or repeated requests don't trigger duplicate downloads, and handles the failure case (no internet) gracefully by still calling the callback, which then shows a "chart unavailable" fallback instead of crashing.

29. app.js Part 7 — The Today (Home) View

`renderToday()` is the single most complex render function, and a good example of composing several smaller render functions together:

```
function renderToday(){
  const totals = dayTotals(currentDate);
  ...
  const macroRows = [
    barRow('Protein', totals.protein, g.proteinGoal, 'g'),
    barRow('Carbs', totals.carbs, g.carbGoal, 'g'),
    barRow('Fat', totals.fat, g.fatGoal, 'g'),
  ].join('');

  const mealSections = STATE.mealSlots.map(slot=>{
    const items = day[slot.id] || [];
    const rows = items.length ? items.map((entry, idx) => `...`).join('') : `<div class="meal-empty">...</div>`;
    return `<div class="meal-card">...${rows}...</div>`;
  }).join('');

  return `
    ${dateNavHtml()}
    <div class="card">
      ${ringSvg(kcalPct, fillColor)}
      <div class="macro-rows">${macroRows}</div>
    </div>
    ${waterCardHtml()}
    ${workoutCardHtml()}
    ${mealSections}
  `;
}
```

Every piece — the date navigator, the calorie ring, the macro bars, the water card, the workout card, each meal card — is its own small function returning its own HTML fragment. `renderToday()`'s only job is to call each one and stitch the results together with template literals. This mirrors "component composition" from frameworks, just without any framework machinery behind it — a function returning a string *is* the component.

29.1 The SVG progress ring

```
function ringSvg(pct, fillColor){
  const size=200, cx=size/2, cy=size/2, r=84;
  const circumference = 2*Math.PI*r;
  const p = clamp(pct,0,1);
  const dash = circumference*p;
  return `<svg viewBox="0 0 ${size} ${size}" style="transform:rotate(-90deg);">
    <circle cx="${cx}" cy="${cy}" r="${r}" fill="none" stroke="var(--surface-2)" stroke-width="16"/>
    <circle cx="${cx}" cy="${cy}" r="${r}" fill="none" stroke="${fillColor}" stroke-width="16"
      stroke-dasharray="${dash} ${circumference}"/>
  </svg>`;
}
```

This is a classic SVG trick for drawing a progress ring without any image or canvas drawing: draw a full circle twice — once as a plain gray "track", then again on top with a dashed stroke pattern where the first "dash" length equals *percentage complete* $\times$ *the circle's full circumference* and the second number is the

remainder, which SVG then renders as one continuous colored arc covering exactly that percentage of the circle. Rotating the whole SVG -90 degrees makes the arc start from the top (12 o'clock) instead of SVG's default 3 o'clock starting point.

30. app.js Part 8 — The Food Picker Flow

Adding a food is a small multi-step "wizard" flow, entirely modeled as one JavaScript object, `picker`, that gets read and mutated as the user moves through it:

```
function openPicker(opts){
  picker = Object.assign({
    step:'list', query:'', cat:'All', meal:null, dateKey:currentDate,
    cart: [], activeRef:null, activeUnit:'medium', activeAmount:1, ...
  }, opts||{});
  renderPickerModal();
}
function renderPickerModal(){
  if(!picker) return;
  const html = picker.step === 'list' ? renderListStep() : renderServingStep();
  openModalHtml(html, 'generic');
}
```

`picker.step` is a simple state machine with two states: `'list'` (search and browse foods) and `'serving'` (pick a quantity for one selected food). `renderPickerModal()` just checks which step it's in and calls the matching render function — the exact same "render function reads state, returns HTML" pattern from Chapter 28, scoped down to a modal instead of a whole page.

The picker also supports a shopping-cart-like flow: tapping a food doesn't immediately log it — it walks you to the serving-size step, then back to the list (with that food now shown in a running "Added so far" cart), so you can add several foods to one meal in a row before finally hitting "Finish" to commit them all to the diary at once:

```
if(action === 'picker-finish'){
  const meal = picker.meal || STATE.mealSlots[0].id;
  const day = getDay(picker.dateKey);
  picker.cart.forEach(item=>{
    const base = resolveRefBase(item.ref);
    const qty = base && base.servingGrams>0 ? item.grams/base.servingGrams : 1;
    day[meal].push({ref:item.ref, qty});
  });
  saveState(); closeModal();
  renderView();
}
```

The same underlying picker object is reused for several related flows by just changing its starting options: `singleMode` (skip the cart, add immediately — used by the Foods tab's "quick add"), editing an existing entry (pre-fill `editMeal/editIndex` so "Add" becomes "Save" and replaces the old entry instead of creating a new one), and `linkOnly` (used when linking a scanned barcode to an existing food, Chapter 34, where picking a food should link it rather than log it).

31. app.js Part 9 — Custom Foods & Custom Meals

A custom food form dynamically generates one input field per nutrient, reusing `NUTRIENT_GROUPS` from Chapter 23 so the form's organization exactly matches the Nutrients tab's organization:

```
function customFoodNutrientFieldsHtml(){
  return NUTRIENT_GROUPS.map(grp=>{
    const rows = grp.rows.filter(([key])=> !CUSTOM_FOOD_PRIMARY_KEYS.includes(key));
    ...
    return `<details class="settings-group"><summary>${grp.title}</summary>${pairsHtml}</details>`;
  }).join('');
}
```

(`<details></summary>` is a native HTML element pair that creates a collapsible section with zero JavaScript needed — clicking the summary text toggles it open/closed automatically, which is why this appears nowhere in the event-handling chapter.) Calories, protein, carbs, and fat are excluded from this generated list because they're always shown as separate, prominent fields at the top of the form — everything else (all ~40 vitamins/minerals/fat-subtypes) is generated.

```
function handleCustomFoodSubmit(form){
  const fd = new FormData(form);
  const name = (fd.get('name')||'').toString().trim();
  if(!name){ toast('Please name this food.');
```

`return;``}`

```
  const num = (k, d)=>{ const v = parseFloat(fd.get(k)); return isNaN(v) ? d : v; };
  const food = { id: 'custom-'+uid(), name, category:'Custom', ... };
  NUTRIENT_KEYS.forEach(k=> food[k] = num(k, 0));
  STATE.customFoods.push(food);
  saveState(); closeModal(); renderView();
}
```

`FormData` is a built-in browser API that reads every named input inside a `<form>` at once, keyed by each input's name attribute — much simpler than manually querying and reading dozens of individual input elements. The small inline helper `num(k, d)` parses a field as a number and falls back to a default `d` if it's empty or invalid, so a nutrient field left blank quietly becomes 0 rather than NaN corrupting later math.

31.1 Custom meals (recipes)

A custom meal is built interactively: search for an ingredient, tap it to add it to the recipe, repeat, then save. Recall from Chapter 25.4 that a meal reference resolves recursively — so once saved, a custom meal behaves exactly like any other loggable food everywhere else in the app, including being usable as an ingredient in searches, but not nestable inside another custom meal (the picker explicitly filters meal-type items out of the ingredient search to prevent that).

32. app.js Part 10 — Progress View & Charts

The weight-trend chart is a thin wrapper around the Chart.js library introduced in Chapter 14 and lazy-loaded in Chapter 28.2:

```
function renderWeightChartNow(){
  const canvas = document.getElementById('weightChart');
  if(typeof Chart === 'undefined'){ /* show fallback text instead */ return; }
  const log = [...STATE.weightLog].sort((a,b)=> a.date < b.date ? -1 : 1);
  const labels = log.map(w=>w.date.slice(5));
  const data = log.map(w=> round1(toDisplayWeight(w.kg)));
  if(weightChart){ weightChart.destroy(); weightChart = null; } // clean up the old chart first!
  weightChart = new Chart(canvas.getContext('2d'), {
    type:'line',
    data:{ labels, datasets:[ {label:'Weight', data, ...}, ...(goal!=null ? [{label:'Goal', ...}] : []) ] },
    options:{ responsive:true, ... }
  });
}
```

Two details worth knowing for an interview: (1) `weightChart.destroy()` is called before creating a new chart instance — Chart.js attaches its own internal event listeners and canvas state to each chart object, and failing to destroy the old one before replacing it (which happens every time this view re-renders) would leak memory and cause visual glitches over time; and (2) the optional goal-line dataset is added conditionally using the spread-into-array trick from Chapter 8.3 — `...(goal!=null ? [{...}] : [])` spreads in either one extra dataset object or nothing at all, cleanly avoiding an `if` statement in the middle of an object literal (which isn't syntactically possible in JavaScript).

The chart's colors are read directly from the current CSS variables (via the `css()` helper from Chapter 23) so the chart automatically matches whichever theme — light or dark — is currently active, without any separate "chart theme" configuration.

33. app.js Part 11 — Profile View

The Profile view is the largest single render function in the app, but structurally it's just several independent `<form>` elements stacked together — each with its own `id`, handled separately in `onSubmit` (Chapter 36). A few UI details worth noting:

- Height can be entered in cm or ft/in — internally, the app *always* stores height as centimeters (`heightCm`); the ft/in inputs are converted on the way in and out, so all downstream math (like `calcBMR`) only ever has to deal with one unit.
- Similarly, weight is always stored internally in kilograms, with `toDisplayWeight()` / `fromDisplayWeight()` converting to/from pounds only at the point of display or input, based on the user's chosen unit preference.
- The micronutrient goal-editing section is wrapped in a collapsible `<details>` element (see Chapter 31) since most users won't want to micromanage 40 individual targets.

34. app.js Part 12 — Barcode Scanning

This feature combines three ideas already covered: the `BarcodeDetector` and `MediaDevices` browser APIs (Chapter 14), `async/await` (Chapter 11), and a recursive polling loop using `requestAnimationFrame`:

```
async function startScan(){
  if(!('BarcodeDetector' in window)){ showScanMessage("not supported"); return; }
  try{
    scanDetector = new window.BarcodeDetector({formats});
    scanStream = await navigator.mediaDevices.getUserMedia({video:{facingMode:'environment'}});
    video.srcObject = scanStream;
    await video.play();
    scanning = true;
    scanLoop();
  }catch(err){ showScanMessage("Camera access isn't available here."); }
}

function scanLoop(){
  if(!scanning) return;
  scanDetector.detect(video).then(codes=>{
    if(codes && codes.length){ handleBarcode(codes[0].rawValue); }
    else if(scanning){ requestAnimationFrame(scanLoop); } // check again next frame
  }).catch(()=>{ if(scanning) requestAnimationFrame(scanLoop); });
}
```

`facingMode: 'environment'` requests the phone's rear (outward-facing) camera specifically, since that's the one normally used for scanning things. `requestAnimationFrame` is a browser API that schedules a function to run right before the next screen repaint (typically ~60 times per second) — using it for the scan loop means the app keeps checking each new video frame for a barcode, but automatically pauses this work if the browser tab isn't visible, which is more efficient than a fixed timer.

When a code is found, `handleBarcode()` looks it up in `STATE.barcodeMap` (Chapter 23.4); if it's already linked to a food, it offers to log that food directly; if it's a never-before-seen barcode, it offers to either link it to an existing food or create a brand new custom food for it — after which future scans of that same barcode resolve instantly.

35. app.js Part 13 — Export / Import / Theme

35.1 Backup export

```
async function exportData(){
  const json = JSON.stringify(STATE, null, 2);
  const filename = 'tiffin-backup-'+todayKey()+'.json';
  try{
    if(window.claude && window.claude.use){
      const downloads = await window.claude.use('downloads');
      if(downloads){ await downloads.save({filename, data: json}); return; }
    }
  }catch(e){}
  // fallback: show the JSON in a text box with a "copy" button
  openModalHtml(`<textarea readonly>${escapeHtml(json)}</textarea> ...`, 'generic');
}
```

`JSON.stringify(STATE, null, 2)` — the third argument, 2, tells it to pretty-print with 2-space indentation, making the exported file human-readable if someone opens it directly. The function first tries an optional platform-specific save-file API (available when running inside certain host environments), and falls back to simply displaying the raw JSON text in a read-only textbox with a "copy to clipboard" button when that's not available — always giving the user *some* way to get their data out, regardless of environment.

35.2 Backup import

```
function importFromFile(file){
  const reader = new FileReader();
  reader.onload = ()=>{
    try{
      const parsed = JSON.parse(reader.result);
      const s = defaultState();
      s.goals = mergeKnownFields(DEFAULT_GOALS, parsed.goals); // <-- security, Ch.24.2
      s.profile = mergeKnownFields(defaultProfile(), parsed.profile); // <-- security, Ch.24.2
      ...
      STATE = s;
      saveState(); applyTheme(); renderApp();
    }catch(e){ toast("That file couldn't be read as a Tiffin backup."); }
  };
  reader.readAsText(file);
}
```

This is exactly the loader logic from `loadState()` (Chapter 24), reused for an imported file — and it's the more important of the two places `mergeKnownFields` gets used, since an imported file is more plausibly something a user downloaded, shared, or hand-edited, rather than something the app wrote itself.

35.3 Theme switching

```
function applyTheme(){
  const t = STATE.goals.theme || 'system';
  if(t === 'system') document.documentElement.removeAttribute('data-theme');
  else document.documentElement.setAttribute('data-theme', t);
}
```

This is the JavaScript half of the dark-mode system introduced in Chapter 21.1: setting or removing the `data-theme` attribute on the root `<html>` element is the ONLY thing this function does — every visual consequence is handled entirely by the CSS variable rules already defined in `app.css` reacting to that attribute changing.

36. app.js Part 14 — The Event System (the app's heart)

Everything explained so far describes functions that *could* run — this section is what actually decides *when* they run. Recall the event delegation pattern from Chapter 10.1; this is its full, real implementation.

```
function onClick(e){
  const t = e.target.closest('[data-action]');
  if(!t) return;
  const action = t.dataset.action;

  if(action === 'nav'){ currentView = t.dataset.view; renderApp(); return; }
  if(action === 'date-prev'){ currentDate = addDaysKey(currentDate, -1); renderView(); return; }
  if(action === 'open-picker'){ openPicker({meal:t.dataset.meal}); return; }
  if(action === 'delete-entry'){
    e.stopPropagation();
    const meal = t.dataset.meal, idx = parseInt(t.dataset.index,10);
    getDay(currentDate)[meal].splice(idx,1);
    saveState(); renderView();
    return;
  }
  // ... roughly 60 more branches, covering every button in the app
}
```

This single function is around 260 lines long and handles essentially every click in the entire application — nav switching, date navigation, opening every modal, every delete button, every settings toggle, the whole food picker flow, barcode scanning controls, meal slot management, and more. Each branch follows an almost identical shape: read some data out of the clicked element's dataset (the collection of its data-* attributes, per Chapter 2.2), mutate STATE or another module-level variable accordingly, then return immediately — the early return after every branch is what makes an otherwise 60-branch if-chain readable: once one matches, nothing below it is even checked.

`e.stopPropagation()`, seen in the delete-entry branch, prevents this click from also bubbling up and triggering a *different* data-action on a parent element (in this specific case, the delete (x) icon sits inside a row that itself has its own `data-action="edit-entry"` for opening the edit screen — without `stopPropagation()`, clicking delete would also immediately trigger edit).

36.1 Three sibling listeners for three other event types

```

function onInput(e){    // fires on every keystroke (live search-as-you-type, live nutrient preview)
    const el = e.target;
    if(el.id === 'foodsSearch'){ foodsFilter.query = el.value; /* re-render just the list */ return; }
    if(el.id === 'servingAmount'){ /* recompute and redraw just the nutrition preview numbers */ return; }
    ...
}
function onChange(e){    // fires when a <input type="file"> selection is confirmed
    if(el.id === 'importFileInput' && el.files[0]){ importFromFile(el.files[0]); return; }
}
function onSubmit(e){    // fires when any <form> is submitted
    if(form.id === 'customFoodForm'){ e.preventDefault(); handleCustomFoodSubmit(form); return; }
    if(form.id === 'profileForm'){ e.preventDefault(); /* save name */ return; }
    if(form.id === 'weightForm'){ e.preventDefault(); /* validate + save a weigh-in */ return; }
    ...
}

```

`e.preventDefault()` inside every form handler stops the browser's default behavior for a form submission — which is to reload the entire page and send the data to a server via a URL, exactly what a no-backend app must never let happen. Note also that `onInput`'s live-search handler deliberately updates *only* the small results-list element (`document.getElementById('foodsList').innerHTML = ...`) rather than calling the full `renderView()` — re-rendering the entire page on every single keystroke of a search box would be needlessly wasteful and could even steal focus away from the input the user is actively typing in.

37. app.js Part 15 — Startup

```
let initied = false;
function init(){
  if(initied) return; initied = true;
  applyTheme();
  document.addEventListener('click', onClick);
  document.addEventListener('input', onInput);
  document.addEventListener('change', onChange);
  document.addEventListener('submit', onSubmit);
  document.getElementById('modalOverlay').addEventListener('click', function(e){
    if(e.target.id === 'modalOverlay') closeModal(); // click OUTSIDE the modal box closes it
  });
  renderApp(); // the very first paint of the whole app
}
document.addEventListener('DOMContentLoaded', init);
if(document.readyState !== 'loading') init();
```

This is the last thing in the file, and it's where everything described in this entire part actually kicks off: apply the saved theme, attach the four delegated event listeners from Chapter 36 exactly once, wire up "click the dark overlay behind a modal to close it," and finally call `renderApp()` for the very first time, painting the initial screen from whatever STATE was loaded at the very top of the file (Chapter 24).

The `initied` guard flag exists because of the two lines at the very bottom: `DOMContentLoaded` fires once, when the browser finishes parsing the HTML — but if the script happens to load and execute *after* that event already fired (a real possibility depending on exactly how/when the script tag runs), the listener would simply never fire at all, and the app would never start. The second line, checking `document.readyState`, catches that case by calling `init()` immediately if the document has already finished loading. The guard flag ensures that if *both* paths somehow end up running, `init()` still only ever truly executes once.

PART 3

Architecture & Design Decisions

38. Why No Framework, No Backend?

38.1 No framework (no React / Vue / Angular)

This app achieves everything a framework normally provides — a render-on-state-change loop, reusable "components," event handling — using nothing but the patterns from Chapters 28 and 36: template literals for components, one delegated listener for events, and full-innerHTML-replacement for updates.

- **Pro:** Zero dependencies, zero build step needed to develop (only to bundle for shipping), and a single ~1,700-line file that a reader can genuinely hold in their head — nothing hides behind an abstraction layer or framework "magic."
- **Con:** No fine-grained DOM diffing (a framework only updates the exact DOM nodes that actually changed; this app replaces whole chunks of HTML every time), no built-in component reuse/composition tooling, and the burden of avoiding bugs like memory leaks (Chapter 32's chart cleanup) falls entirely on the developer instead of the framework.
- **Why it's a reasonable choice here:** the app's total data size is small (one user's diary, not millions of rows), so the performance cost of "just re-render everything" never becomes noticeable, and the simplicity payoff — no build tooling required to even run the app locally, trivial to audit for security since every line of logic is visible in one file — is worth it for a project this size.

38.2 No backend, no server, no database

Every user's data lives only in their own browser's localStorage (Chapter 13). There is no login, no account, no API calls that send personal data anywhere.

- **Pro:** Perfect privacy by construction (nothing to breach — there's no server holding anyone's data), zero server hosting cost, and the app works fully offline once loaded, since even the food database is baked directly into the page (Chapter 22).
- **Con:** No cross-device sync (your diary on your phone and your laptop are two separate, unconnected datasets), and clearing browser data destroys everything unless a backup was manually exported (Chapter 35).
- **Why it's a reasonable choice here:** a personal nutrition tracker's core value doesn't strictly require sync or accounts, and trading that away buys meaningful simplicity and privacy.

39. Security: XSS, CSP, Prototype Pollution

39.1 XSS (Cross-Site Scripting) and escapeHtml

Because so much of this app's HTML is built by directly interpolating data into template literals (``<div>${food.name}</div>``), there's a real risk: if `food.name` ever contained something like `<img src=x onerror="steal_data()">` — for instance, typed in by the user themselves when naming a custom food — and it were inserted unescaped, the browser would parse it as real HTML and execute that injected script. This class of vulnerability is called **XSS (Cross-Site Scripting)**.

```
function escapeHtml(s){
  return String(s).replace(/&<>"'/g, c=>({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'
  }[c]));
}

// used everywhere untrusted text is inserted into HTML:
`<div class="fname">${escapeHtml(food.name)}</div>`
```

`escapeHtml()` converts the characters that give HTML its special meaning (`<` `>` `&` `"` `'`) into their harmless text equivalents ("HTML entities"), so `<img>` becomes the literal, inert text `<img>` instead of being parsed as a tag. This function is called on essentially every single piece of user-controlled or data-sourced text before it's woven into an HTML string anywhere in the app — food names, profile name, custom meal names, barcode numbers, all of it.

39.2 The Content-Security-Policy, revisited

`escapeHtml` is the app's *first* line of defense — but defensive engineering means assuming a bug slips through anyway (a future edit that forgets to call it, say). Recall from Chapter 19.1 that `build.py` computes cryptographic hashes of the app's own two legitimate inline scripts and locks the browser down to *only* executing scripts matching those exact hashes. Even in the worst case — an attacker somehow gets an unescaped, malicious string containing a `<script>` tag into the page — the browser's own CSP enforcement would refuse to execute it, because its content wouldn't match either trusted hash. This is called **defense in depth**: multiple independent layers of protection, so that one layer failing doesn't mean total compromise.

39.3 Prototype pollution, revisited

Covered in full in Chapter 24.2 — `mergeKnownFields()` ensures that loading saved state or an imported backup file can never inject unexpected properties (like `__proto__`) into the app's data, only ever the specific fields the app already knows about and expects.

40. The Full Data Flow, End to End

Putting every earlier chapter together, here is literally everything that happens, in order, when a user adds "1.5 cups of rice" to their lunch:

- **1.** User taps + Add Lunch → the click bubbles to document → `onClick` (Ch.36) matches `data-action="open-picker"` → calls `openPicker({meal: 'meal-2'})` (Ch.30).
- **2.** `openPicker` creates the picker state object and calls `renderPickerModal()`, which calls `renderListStep()` → returns an HTML string → `openModalHtml()` sets `#modalContent.innerHTML` and reveals the modal overlay.
- **3.** User types "rice" → fires an input event → `onInput` (Ch.36.1) updates `picker.query`, calls `filterFoods()` (Ch.22/30) against `FOOD_DB`, and replaces just the results list's `innerHTML`.
- **4.** User taps the "Rice, cooked" result → `onClick` matches `select-food` → sets `picker.activeRef` and `picker.step = 'serving'` → re-renders the modal, now showing `renderServingStep()`'s quantity screen.
- **5.** User sets amount to 1.5 → `onInput` recomputes grams/calories live via `computeGrams()` and `scaleNutrients()` (Ch.25) and updates just the preview numbers on screen.
- **6.** User taps Add to list → `onClick` matches `add-to-cart` → pushes the item into `picker.cart`, returns to the list step.
- **7.** User taps Finish → `onClick` matches `picker-finish` → for each cart item, computes its quantity and `getDay(dateKey)[meal].push({ref, qty})` — this is the moment `STATE` itself actually changes (Ch.26).
- **8.** Still inside that handler: `saveState()` serializes the whole `STATE` object to JSON and writes it to `localStorage` (Ch.24) — the data is now durable, surviving even if the tab is closed immediately.
- **9.** `closeModal()` hides the modal; `renderView()` (Ch.28) calls `renderToday()` (Ch.29) fresh — it calls `dayTotals()` (Ch.26), which now includes the new rice entry, recomputes the calorie ring's percentage, and returns brand-new HTML reflecting the updated total.
- **10.** That HTML replaces `#viewRoot`'s content; the browser repaints; the user sees the ring, the macro bars, and the lunch card all instantly reflect the new food — done.

Note: Every one of those ten steps is just a composition of small, individually simple functions covered earlier in this guide. There's no framework, no network request, no hidden magic anywhere in this chain — which is precisely the point being made in Chapter 38.

PART 4

Interview Guide

You now understand every language this app uses and every significant piece of its code. This part turns that understanding into interview-ready answers. Read the question, try to answer it yourself first, then compare against the model answer.

41. The 30-Second Pitch

"Tiffin is a full-featured nutrition tracker — think Cronometer or MyFitnessPal — built as a single, self-contained, zero-dependency web app in vanilla JavaScript. It tracks 44 nutrients across a 332-food built-in database with an Indian-and-international focus, supports barcode scanning, custom foods and recipes, weight tracking with charts, and BMR/TDEE-based goal calculation using the Mifflin-St Jeor formula. There's no backend and no account system — everything is stored client-side in localStorage, which means it's completely private and works offline. It ships as a website, and also as installable native iOS and Android apps via Capacitor, with both builds automated end-to-end through GitHub Actions."

42. Core Technical Q&A;

Q: Why vanilla JavaScript instead of React or another framework?

A: For a single-user, client-only app of this size, a framework's main benefits — efficient re-rendering of huge, frequently-changing datasets, and managing complex component trees — aren't really needed. The tradeoff favored simplicity: zero build tooling required to run the app locally, one file that's fully auditable top to bottom, and no dependency updates to manage. The app still uses the same core idea frameworks are built on — 'the UI is a pure function of state' — just implemented by hand with template literals and full innerHTML replacement instead of a virtual DOM.

Q: How does the app persist data without a server?

A: It uses the browser's localStorage API, which is a simple key-value store that survives page reloads and browser restarts. The app's entire state — profile, goals, diary, custom foods, weight log — is one JavaScript object, serialized to a JSON string with JSON.stringify and written under one key. On load, it's read back with localStorage.getItem and parsed with JSON.parse. This means all data stays on the user's own device and never leaves it.

Q: Walk me through what happens end to end when a user logs a food.

A: See Chapter 40 for the full ten-step trace — in short: a click is caught by one delegated event listener, which mutates the central STATE object directly, immediately persists it to localStorage, then calls the render function for the current view, which reads the now-updated STATE and returns fresh HTML that replaces what's on screen.

Q: Explain the event delegation pattern used here, and why.

A: Instead of attaching a separate click listener to every button (which could be hundreds across all the screens), the app attaches exactly one listener to `document`. It relies on event bubbling — a click always propagates up through all of its ancestor elements — so a single top-level handler can inspect `event.target.closest('[data-action]')` to find out what was actually clicked, then dispatch based on that element's `data-action` attribute. This scales to any number of dynamically-created buttons with zero extra listener setup, which matters a lot here since almost the entire UI is regenerated from scratch on every state change.

Q: What security measures does this app take, and why?

A: Three layers: (1) escapeHtml() sanitizes any data-derived text before it's inserted into an HTML template, preventing XSS if a food name or similar field ever contained malicious markup; (2) a strict Content-Security-Policy, computed at build time from SHA-256 hashes of the app's own two legitimate inline scripts, means the browser will refuse to execute any script that doesn't match — a defense-in-depth backstop even if an escaping bug slipped through; (3) mergeKnownFields() guards against prototype pollution (CWE-1321) when loading saved or imported JSON data, by only ever copying keys the app already expects, rather than blindly merging an untrusted object's keys, which could otherwise let a key like `\_\_proto\_\_` corrupt the behavior of every object in the running app.

Q: How does the app support both light and dark mode?

A: Through CSS custom properties (variables). The entire color palette is defined once as variables under `:root`, and every other CSS rule references them via `var(--name)` instead of hardcoding colors. A dark-mode block simply redefines the same variable names with dark values, either automatically (via the `prefers-color-scheme` media query, following the OS setting) or explicitly (via a `data-theme` attribute the app sets on based on the user's in-app choice). No individual CSS rule needs to know or branch on which theme is active.

Q: How is the mobile app (iOS/Android) built from the same code as the website?

A: Capacitor wraps the built web app (a single static HTML file produced by `build.py`) inside a native app shell for each platform — essentially a full-screen native `WebView` pointed at that file. The same JavaScript that runs in a desktop browser tab runs unmodified inside it. Two GitHub Actions workflows automate the whole pipeline on every push: build the web app, run ``cap sync`` to copy it into the native project, then run each platform's own native compiler (Gradle for Android, Xcode for iOS) to produce an installable `.apk` or `.ipa` file as a downloadable build artifact.

Q: Why does the app store references to foods (type + id) in the diary instead of copies of the nutrition data?

A: Two reasons: it keeps the diary lightweight, and — more importantly — it means editing a custom food's nutrition values later automatically updates every past diary entry that referenced it, since the nutrients are looked up fresh (`resolveRefBase`) every time totals are computed, rather than being frozen at the moment they were logged.

Q: What's a tradeoff of the 'replace the whole innerHTML on every change' rendering approach?

A: It's simple and easy to reason about, but it's not fine-grained — even a change affecting one small part of the screen regenerates and replaces the whole view's HTML. For an app of this size and a single user's dataset, that's imperceptibly fast on modern devices, so it's a reasonable trade for the simplicity gained. A framework with a virtual DOM would only touch the specific DOM nodes that actually changed, which matters more at larger scale.

43. Deep-Dive Design Questions

Q: Why is the food data (NUTRIENT_KEYS) represented as a list of strings that other code loops over, instead of just writing out each nutrient calculation by hand?

A: It turns what would be ~44 nearly-identical lines of code (per function) into one small loop, in every function that touches nutrients: zeroing them out, scaling by quantity, adding two nutrient sets together. It also means the same list drives the Nutrients tab's display grouping and the custom-food form's generated fields — add a 45th nutrient to that one list, and (with matching data in `food_db.json`) it flows through the whole app automatically, rather than needing edits in a dozen different places.

Q: Why does `calcBMR()` return null instead of, say, 0 or throwing an error, when profile data is missing?

A: Returning null makes 'no answer is available yet' an explicit, checkable state, distinct from a legitimate calculated value (0 would be misleading — nobody's BMR is actually zero). Every caller checks `if (bmr == null)` and shows an appropriate fallback (like a prompt to fill in Profile) instead of either crashing or silently displaying a wrong number.

Q: Why build a custom HTML file with 4 placeholder substitutions in Python, instead of using a real bundler like Webpack or Vite?

A: The whole app deliberately has no npm-based build dependencies for its JavaScript — it's plain, unbundled, unminified JS designed to be read and audited directly. A real bundler would be overkill for combining exactly four static files into one; the ~85-line Python script does that one job directly, including computing the CSP script hashes a general-purpose bundler wouldn't know to do out of the box, with no extra dependencies of its own.

Q: What would you change if this app needed to support multiple devices for the same user?

A: The core blocker is that `localStorage` is local to one browser on one device — you'd need some remote storage a user's various devices could all read and write, which means introducing a backend and some form of authentication, likely a real API rather than the current 'no server' model. You'd also need a strategy for merging changes made offline on two devices (a sync/conflict-resolution problem), which the current single-device, single-writer model avoids entirely.

Q: How would you test this codebase?

A: The pure, render-function-per-view architecture (Ch.28) is actually quite testable in principle: most render functions are pure functions of STATE that return a string, so you could feed them a known STATE object and assert on the returned HTML string without needing a real browser. The nutrient math functions (`scaleNutrients`, `addNutrients`, `dayTotals`) are pure and trivially unit-testable. The harder parts to test are the event-delegation handlers, since they mix DOM reads, STATE mutation, and re-rendering together — those would benefit most from a browser-based testing tool (like Playwright) driving real clicks and asserting on the resulting DOM, since they're what's not currently covered.

44. Glossary — Every Term Used in This Guide

API — A set of functions/rules that let one piece of software talk to another.

Array — An ordered list of values, e.g. [1, 2, 3].

Async/await — JavaScript syntax for writing code that waits on slow operations without freezing the page.

Attribute — Extra information on an HTML tag, written as name="value".

Bundler — A build tool (e.g. Webpack, Vite) that combines many source files into fewer files for shipping.

CDN — Content Delivery Network — a server that hosts common public files (like libraries) for fast loading.

CI/CD — Continuous Integration/Deployment — automatically building/testing code on every change.

CSP — Content-Security-Policy — a browser rule set restricting what scripts/styles are allowed to run.

CSS — Cascading Style Sheets — the language that styles HTML's appearance.

CWE — Common Weakness Enumeration — a standard catalog of software vulnerability types.

Callback — A function passed into another function, to be called later.

Closure — A function that remembers variables from the scope it was defined in, even after that scope ends.

Commit — A saved snapshot of a project's files in Git, with a description of what changed.

Component — A reusable, self-contained piece of UI (here: a function returning an HTML string).

DOM — Document Object Model — the browser's live, in-memory tree of a page's elements.

Data attribute — A custom HTML attribute starting with data-, readable by JavaScript.

Destructuring — JS syntax for unpacking values from an array/object into separate variables.

Event — Something that happens on a page: a click, keystroke, form submit, etc.

Event bubbling — A DOM behavior where an event fires on an element, then its parent, then its parent's parent, and so on.

Event delegation — Listening for events on one shared ancestor instead of each individual element.

Flexbox — A CSS layout system for arranging elements in a single row or column.

Function — A named, reusable block of code.

Grid — A CSS layout system for arranging elements in rows and columns at once.

HTML — HyperText Markup Language — describes a page's structure using tags.

IIFE — Immediately Invoked Function Expression — a function defined and called in one step, to create an isolated scope.

JSON — JavaScript Object Notation — a text format for structured data.

Object — A collection of named key-value pairs, e.g. {name: 'Alice'}.

Prototype pollution — A vulnerability class (CWE-1321) where untrusted data corrupts a shared object prototype.

Render — To convert data into HTML/UI and display it.

SHA-256 — A cryptographic function that produces a unique fingerprint (hash) of some data.

Selector — A CSS pattern (like .class or #id) that picks which elements a rule applies to.

Single Page App (SPA) — A web app that updates its content dynamically in place instead of loading new full HTML pages.

State — The current data driving what an app displays.

String — Text data, e.g. "hello".

Template literal — A backtick-quoted JS string that can embed expressions via \${...}.

Ternary operator — A compact if/else expression: condition ? valueIfTrue : valueIfFalse.

Truthy/falsy — JavaScript's rules for treating non-boolean values as true or false in a condition.

Web API — A built-in browser capability beyond basic HTML/CSS/JS, e.g. camera access or clipboard access.

XSS — Cross-Site Scripting — a vulnerability where untrusted text is executed as HTML/script.

YAML — A simple, indentation-based text format often used for configuration files.

const — A JavaScript variable declaration that can't be reassigned.

f-string — Python's syntax for embedding variables directly in a string, e.g. f"Hi {name}".

let — A JavaScript variable declaration that CAN be reassigned later.

localStorage — A browser API for storing key-value data that persists across sessions.

Prepared: September 17, 2026 | **Project:** Tiffin — Nutrition Tracker | **Stack:** HTML, CSS, JavaScript, JSON, Python (build), Capacitor (mobile), GitHub Actions (CI/CD)